

```
{
  "creator" : "ari.neko@cvut.cz",
  "timestamp" : "09-07-2026_17:25:40",
  "variant" : 3,
  "subjectName" : "java-programming",
  "categories" : [ "Exception Handling", "Generics & Types", "Multi-threading & Concurrency" ],
  "questions" : [ {
    "question" : "What happens if a catch block intercepts an exception and rethrows a new
    RuntimeException wrapper?",
    "answers" : [ "The JVM defaults all text inputs back to empty strings", "The current execution path
    breaks and the exception propagates up the thread stack frame", "The active relational database
    transaction commits its row status", "The operating system kernel terminates the parent thread
    launcher instance" ],
    "correctIndex" : 1
  }, {
    "question" : "What type of exception is a NullPointerException classified as inside the Java runtime
    architecture?",
    "answers" : [ "An Error exception condition", "An Unchecked Exception (RuntimeException)", "A
    Compile-time syntax flag anomaly", "A Checked Exception" ],
    "correctIndex" : 1
  }, {
    "question" : "What does the wildcard bound pattern 'List<? extends Number>' accomplish inside a
    method signature?",
    "answers" : [ "It restricts the parameter to a List containing Number or any of its subclasses", "It
    mandates that all elements in the List must carry matching static values", "It turns the List into an
    immutable collection", "It opens the List to accept additions of any custom object structure" ],
    "correctIndex" : 0
  }, {
    "question" : "What happens when you use a raw type instead of a parameterized generic type
    declaration?",
    "answers" : [ "You lose compile-time type safety checking benefits and trigger compiler warning
    logs", "The Java Virtual Machine executes the bytecode lines 50 percent slower", "Jackson's
    ObjectMapper becomes unable to parse JSON resource file streams", "The application crashes
    instantly with a ClassCastException upon boot paths" ],
    "correctIndex" : 0
  }, {
    "question" : "What does the wildcard pattern 'List<? super Integer>' signify inside type boundaries
    checking?",
    "answers" : [ "It specifies a List that can hold Integer or any of its master superclass ancestor
    layers", "It prevents downstream functions from executing list mutations", "It tells the system that
    the collection properties are bound to an index", "It limits the collection strictly to elements that
    implement serializable" ],
```

```

"correctIndex" : 0
}, {
"question" : "What primary objective does Java's Type Erasure accomplish when compiling generic code architectures?",
"answers" : [ "It replaces type parameters with bounds or Object to ensure type safety without runtime overhead", "It restricts collection instances from scaling horizontally", "It forces the compiler to duplicate class files for every unique type", "It converts object reference addresses into encrypted byte arrays" ],
"correctIndex" : 0
}, {
"question" : "What structural capability advantage does a Callable task possess compared to a standard Runnable task?",
"answers" : [ "A Callable does not require module-info reflection opens configurations", "A Callable runs directly inside kernel privilege rings bypassing restrictions", "A Callable can be restarted multiple times within the same thread lifecycle", "A Callable can return a computed result value and throw checked exceptions" ],
"correctIndex" : 3
}, {
"question" : "What occurs if an execution thread invokes .wait() on an object without holding that object's monitor lock?",
"answers" : [ "The underlying database statement execution rolls back its current update", "It throws an immediate IllegalMonitorStateException at runtime", "The thread drops into a safe background daemon sleep loop", "The context switch routine allocates more processing cycles" ],
"correctIndex" : 1
}, {
"question" : "Which thread-safe collection from java.util.concurrent should replace a standard HashMap in high-concurrency environments?",
"answers" : [ "Hashtable", "LinkedHashMap module structure", "ConcurrentHashMap", "Collections.synchronizedMap" ],
"correctIndex" : 2
}, {
"question" : "What thread safety violation occurs when a thread caches local variable copies instead of reading from main RAM?",
"answers" : [ "Deadlock layout lock", "Illegal Thread State condition", "Race Condition anomaly", "Memory Visibility Problem" ],
"correctIndex" : 3
} ]
}

```